

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA ĐIỆN – ĐIỆN TỬ
Bộ Môn: Điều Khiển Học



BÁO CÁO HỌC PHẦN
HỆ THỐNG ĐIỀU KHIỂN ROBOT CÔNG NGHIỆP

Họ và tên : Nguyễn Mạnh Cường

MSV: : 213310409

Lớp : Kỹ thuật Robot và Trí tuệ nhân tạo K62

Giảng viên : PGS.TS. Võ Thanh Hà

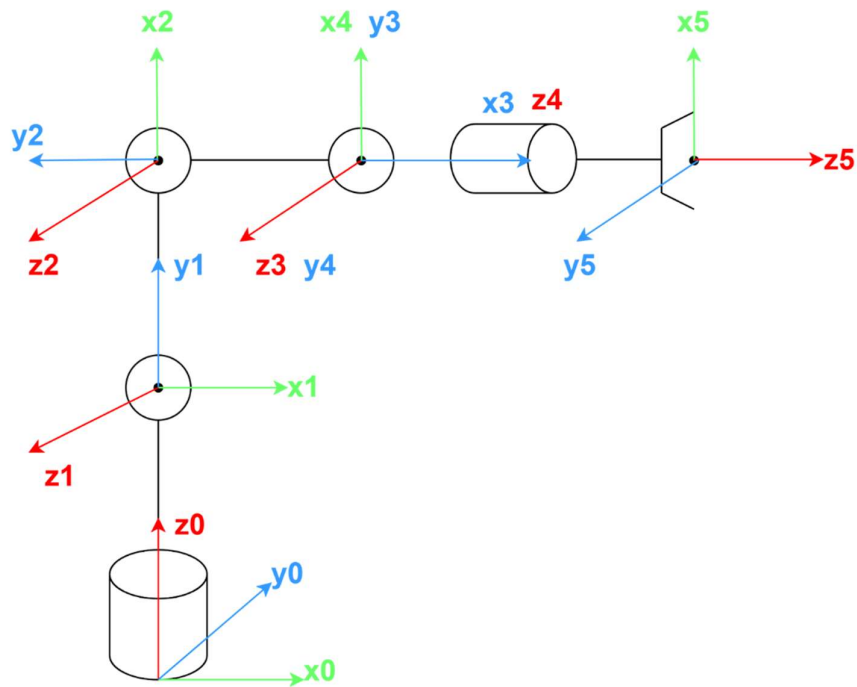
MỤC LỤC

PHẦN I: Thiết kế mô hình động học tay máy 6 bậc tự do thực hiện thao tác xếp hộp vào pallet theo ma trận 3x3	3
I. Động học robot	3
II. Điều khiển PID.....	7
III. Mô phỏng thực hiện thao tác xếp hộp vào pallet theo ma trận 3x3.....	12
PHẦN II: URSim: lập trình UR3 gấp vật và xếp thành hàng – cột.....	15
I. Giới thiệu robot UR3 và bài toán	15
II. Cấu hình chuyển động robot.....	15
PHẦN III: RobotStudio: mô phỏng thực hiện nhiệm vụ xếp pallet.	23
I. Mục tiêu hệ thống.....	23
II. Mô hình mô phỏng RobotStudio	23
III. Thuật toán và chương trình RAPID	28
IV. Thuật toán xử lý ảnh & chương trình Python	31
V. Quy trình hoạt động tổng thể	33

PHẦN I: Thiết kế mô hình động học tay máy 6 bậc tự do thực hiện thao tác xếp hộp vào pallet theo ma trận 3x3

I. Động học robot

1.1. Đặt hệ trục tọa độ



1.2. Bảng thông số Denavit–Hartenberg (DH)

Khâu i	a_i	α_i	d_i	θ_i
1	0	90°	d_1	θ_1
2	a_2	0°	0	θ_2
3	a_3	0°	0	θ_3
4	0	90°	0	θ_4
5	0	0°	d_4	θ_5

1.3. Công thức ma trận biến đổi

$$T_i^{i-1} = \begin{pmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & \alpha_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & \alpha_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

1.4. Các ma trận biến đổi của từng khâu

$$T_1^0 = \begin{pmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad T_2^1 = \begin{pmatrix} c_2 & -s_2 & 0 & a_2 c_2 \\ s_2 & c_2 & 0 & a_2 s_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$T_3^2 = \begin{pmatrix} c_3 & -s_3 & 0 & a_3 c_3 \\ s_3 & c_3 & 0 & a_3 s_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad T_4^3 = \begin{pmatrix} c_4 & 0 & s_4 & 0 \\ s_4 & 0 & -c_4 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$T_5^4 = \begin{pmatrix} c_5 & -s_5 & 0 & 0 \\ s_5 & c_5 & 0 & 0 \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Ma trận biến đổi tổng thể

$$T_5^0 = T_1^0 \cdot T_2^1 \cdot T_3^2 \cdot T_4^3 \cdot T_5^4 = \begin{pmatrix} c_1 \cdot c_{234} \cdot c_5 + s_1 s_5 & -c_1 \cdot c_{234} \cdot s_5 + s_1 \cdot c_5 & c_1 \cdot s_{234} & c_1 (s_{234} \cdot d_4 + a_3 \cdot c_{23} + a_2 \cdot c_2) \\ s_1 \cdot c_{234} \cdot c_5 - c_1 s_5 & s_1 \cdot c_{234} \cdot s_5 - c_1 \cdot c_5 & s_1 \cdot s_{234} & s_1 (s_{234} \cdot d_4 + a_3 \cdot c_{23} + a_2 \cdot c_2) \\ s_{234} \cdot c_5 & -s_{234} \cdot s_5 & -c_{234} & -c_{234} \cdot d_4 + s_{23} \cdot a_3 + a_2 \cdot s_2 + d_1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Hướng (ma trận quay) của end-effector

$$R = \begin{bmatrix} c_1 \cdot c_{234} \cdot c_5 + s_1 s_5 & -c_1 \cdot c_{234} \cdot s_5 + s_1 \cdot c_5 & c_1 \cdot s_{234} \\ s_1 \cdot c_{234} \cdot c_5 - c_1 s_5 & s_1 \cdot c_{234} \cdot s_5 - c_1 \cdot c_5 & s_1 \cdot s_{234} \\ s_{234} \cdot c_5 & -s_{234} \cdot s_5 & -c_{234} \end{bmatrix}$$

- Vị trí end-effector

$$P = \begin{bmatrix} P_x \\ P_y \\ P_z \end{bmatrix} = \begin{bmatrix} c_1 (s_{234} \cdot d_4 + a_3 \cdot c_{23} + a_2 \cdot c_2) \\ s_1 (s_{234} \cdot d_4 + a_3 \cdot c_{23} + a_2 \cdot c_2) \\ -c_{234} \cdot d_4 + s_{23} \cdot a_3 + a_2 \cdot s_2 + d_1 \end{bmatrix}$$

1.5. Thiết lập phương trình động học nghịch

- Động học thuận:

$$T_5^0 = T_1^0 \cdot T_2^1 \cdot T_3^2 \cdot T_4^3 \cdot T_5^4$$

- Suy ra:

$$\left[T_1^0 \right]^{-1} \cdot T_5^0 = T_2^1 \cdot T_3^2 \cdot T_4^3 \cdot T_5^4 \quad (*)$$

- Ta có:

$$\left[T_1^0 \right]^{-1} = \begin{pmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} c_1 & s_1 & 0 & 0 \\ 0 & 0 & 1 & -d_1 \\ s_1 & -c_1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Vế trái của phương trình (*) được tính toán như sau:

$$VT = \left[T_1^0 \right]^{-1} \cdot T_5^0 = \begin{pmatrix} c_1 & s_1 & 0 & 0 \\ 0 & 0 & 1 & -d_1 \\ s_1 & -c_1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$VT = \begin{pmatrix} c_1 \cdot n_x + s_1 \cdot n_y & c_1 \cdot o_x + s_1 \cdot o_y & c_1 \cdot a_x + s_1 \cdot a_y & c_1 \cdot p_x + s_1 \cdot p_y \\ n_z & o_z & a_z & p_z - d_1 \\ s_1 \cdot n_x - c_1 \cdot n_y & s_1 \cdot o_x - c_1 \cdot o_y & s_1 \cdot a_x - c_1 \cdot a_y & s_1 \cdot p_x - c_1 \cdot p_y \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Vế phải của phương trình (*) được tính toán như sau:

$$VP = T_5^2 = T_2^1 \cdot T_3^2 \cdot T_4^3 \cdot T_5^4 = \begin{pmatrix} c_{234} \cdot c_5 & -c_{234} \cdot s_5 & s_{234} & s_{234} \cdot d_4 + a_3 \cdot c_{23} + a_2 \cdot c_2 \\ s_{234} \cdot c_5 & -s_{234} \cdot s_5 & -c_{234} & -c_{234} \cdot d_4 + a_3 \cdot s_{23} + a_2 \cdot s_2 \\ s_5 & c_5 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Thực hiện đồng nhất 2 vế:

- **Tính θ_1 :**

$$\text{Ta có: } VT_{34} = s_1 p_x - c_1 p_y; VP_{34} = 0$$

$$\frac{s_1}{c_1} = \frac{p_y}{p_x} \quad \tan \theta_1 = \frac{p_y}{p_x}$$

$$\Rightarrow \theta_1 = \arctan 2(p_y, p_x)$$

○ **Tính θ_5 :**

$$VP_{31} = s_5; VT_{31} = s_1 n_x - c_1 n_y$$

$$VP_{32} = c_5; VT_{31} = s_1 o_x - c_1 o_y$$

$$s_5 = s_1 n_x - c_1 n_y$$

$$c_5 = s_1 o_x - c_1 o_y$$

$$\Rightarrow \theta_5 = a \tan 2(s_1 n_x - c_1 n_y, s_1 o_x - c_1 o_y)$$

○ **Tính $\theta_2 + \theta_3 + \theta_4$:**

$$VP_{13} = s_{234}; VT_{13} = c_1 a_x - s_1 a_y$$

$$VP_{23} = -c_{234}; VT_{23} = a_z$$

$$s_{234} = c_1 a_x - s_1 a_y; -c_{234} = a_z$$

$$\Rightarrow \theta_2 + \theta_3 + \theta_4 = a \tan 2(c_1 a_x - s_1 a_y, -a_z)$$

○ **Tính θ_3 :**

$$VP_{14} = s_{234} d_4 + a_3 c_{23} + a_2 c_2; VT_{14} = c_1 p_x - s_1 p_y$$

$$VP_{24} = -c_{234} d_4 + a_3 s_{23} + a_2 s_2; VT_{24} = p_z - d_1$$

Đặt:

$$m = c_1 p_x - s_1 p_y - s_{234} d_4 = a_3 c_{23} + a_2 c_2$$

$$n = p_z - d_1 + c_{234} d_4 = a_3 s_{23} + a_2 s_2$$

Bình phương và cộng:

$$m^2 + n^2 = (a_3 c_{23} + a_2 c_2)^2 + (a_3 s_{23} + a_2 s_2)^2 = a_3^2 + a_2^2 + 2a_3 a_2 \cos \theta_3$$

$$\cos \theta_3 = \frac{m^2 + n^2 - a_3^2 - a_2^2}{2a_3 a_2} = k$$

$$\sin \theta_3 = \pm \sqrt{1 - k^2}$$

$$\Rightarrow \theta_3 = a \tan 2(\pm \sqrt{1 - k^2}, k)$$

○ **Tính θ_2 :**

$$c_{23} = c_2 c_3 - s_2 s_3 \quad s_{23} = s_2 c_3 + c_2 s_3$$

$$m = (a_2 c_3 + a_2) c_2 - a_3 s_3 s_2 \quad n = (a_3 c_3 + a_2) s_2 - a_3 s_3 c_2$$

Đặt

$$u = a_3 c_3 + a_2 \quad v = a_3 s_3$$

$$mu + nv = (u^2 + v^2) \cos \theta_2$$

$$mu - nv = (u^2 + v^2) \sin \theta_2$$

$$\Rightarrow \theta_2 = a \tan 2(nu - mv, mu + nv)$$

○ **Tính θ_4 :**

$$\text{Ta có: } \theta_2 + \theta_3 + \theta_4 = a \tan 2(c_1 a_x - s_1 a_y, -a_z)$$

$$\Rightarrow \theta_4 = a \tan 2(c_1 a_x + s_1 a_y, -a_z) - \theta_2 - \theta_3$$

Kết quả:

$$\theta_1 = \text{atan2}(p_y, p_x),$$

$$\theta_5 = \text{atan2}(s_1 n_x - c_1 n_y, s_1 o_x - c_1 o_y),$$

$$\theta_3 = \text{atan2}(\pm \sqrt{1 - k^2}, k), \quad k = \frac{m^2 + n^2 - a_3^2 - a_2^2}{2a_3 a_2},$$

$$\theta_2 = \text{atan2}(nu - mv, mu + nv), \quad u = a_3 c_3 + a_2, \quad v = a_3 s_3,$$

$$\theta_4 = \text{atan2}(c_1 a_x + s_1 a_y, -a_z) - \theta_2 - \theta_3.$$

II. Điều khiển PID

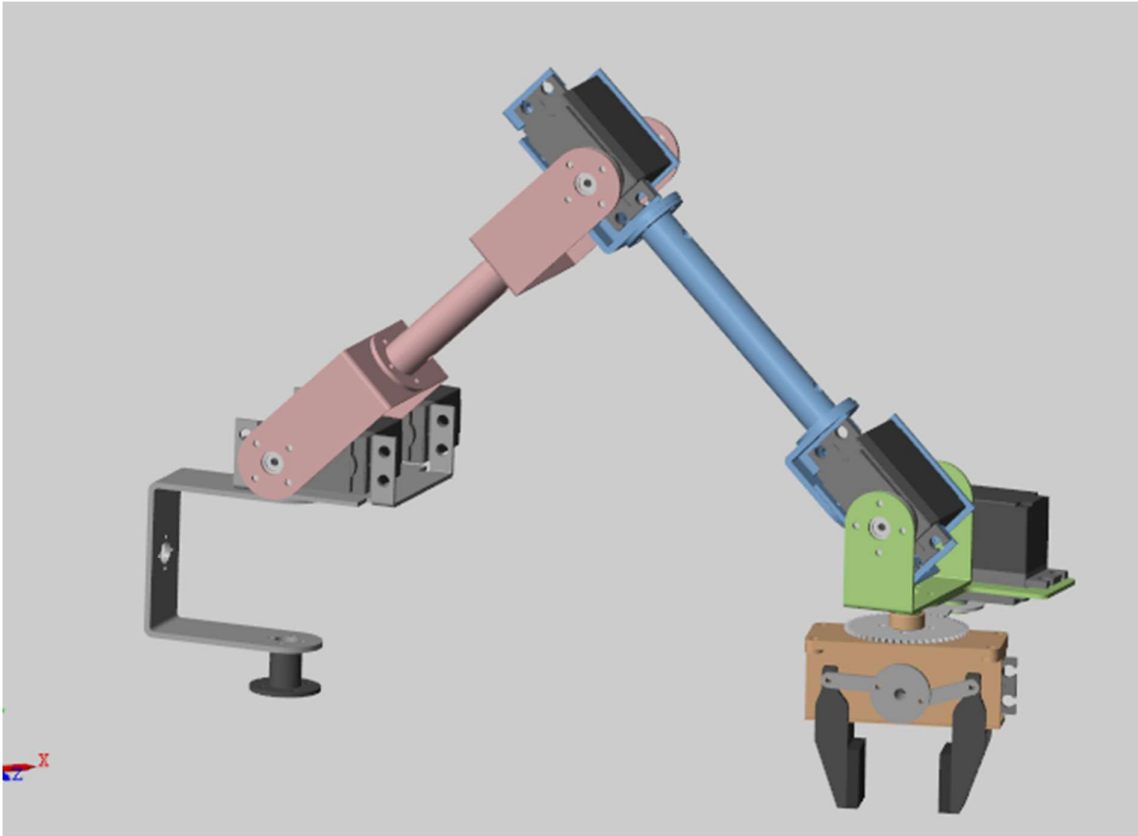
2.1. Giới thiệu & Mục tiêu

- Trong điều khiển robot, bộ điều khiển PID control (Proportional–Integral–Derivative) vẫn là phương pháp phổ biến vì đơn giản, ổn định và dễ thực hiện.
- Tuy nhiên, với một cánh tay robot nhiều khớp (6-DOF), việc “tune” PID cho mỗi khớp một cách đơn lẻ thường không đảm bảo hiệu năng chung — do tương

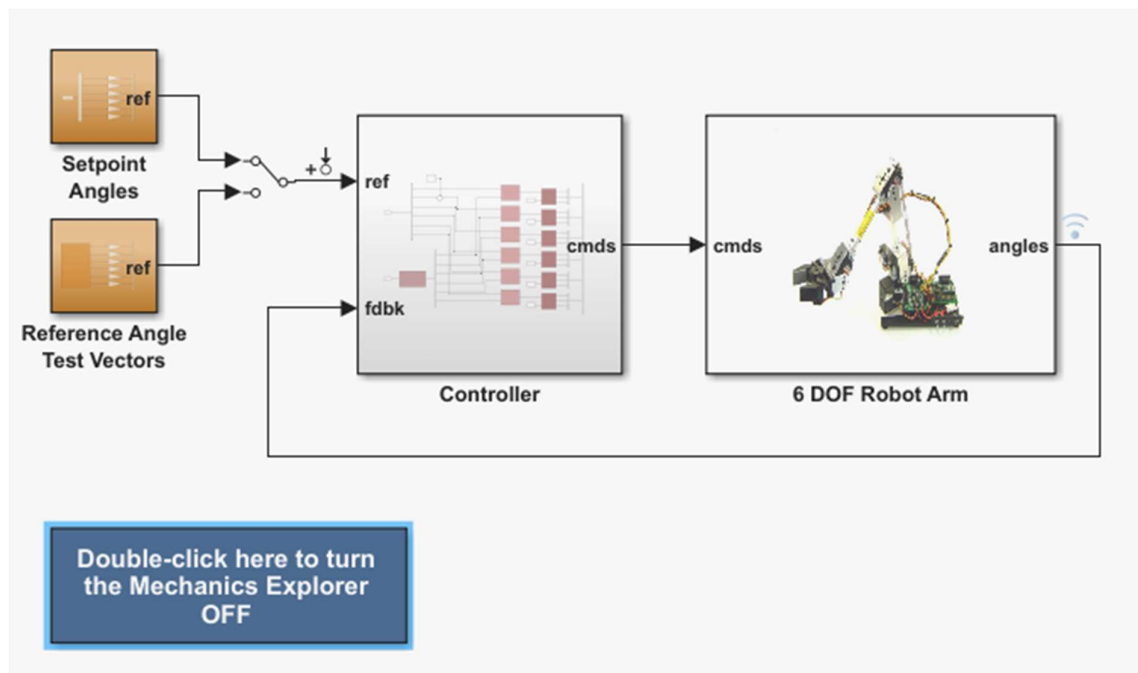
tác động học, coupling giữa các khớp, lực quán tính khớp, mô-men quán tính của liên kết, v.v.

- Mục tiêu của phần này: áp dụng “multi-loop PID tuning” để điều khiển chính xác một robot 6-khớp, đảm bảo theo dõi vị trí góc mục tiêu (joint-angle reference), đạt độ ổn định, đáp ứng nhanh, sai số nhỏ.

2.2. Mô hình Robot & Controller

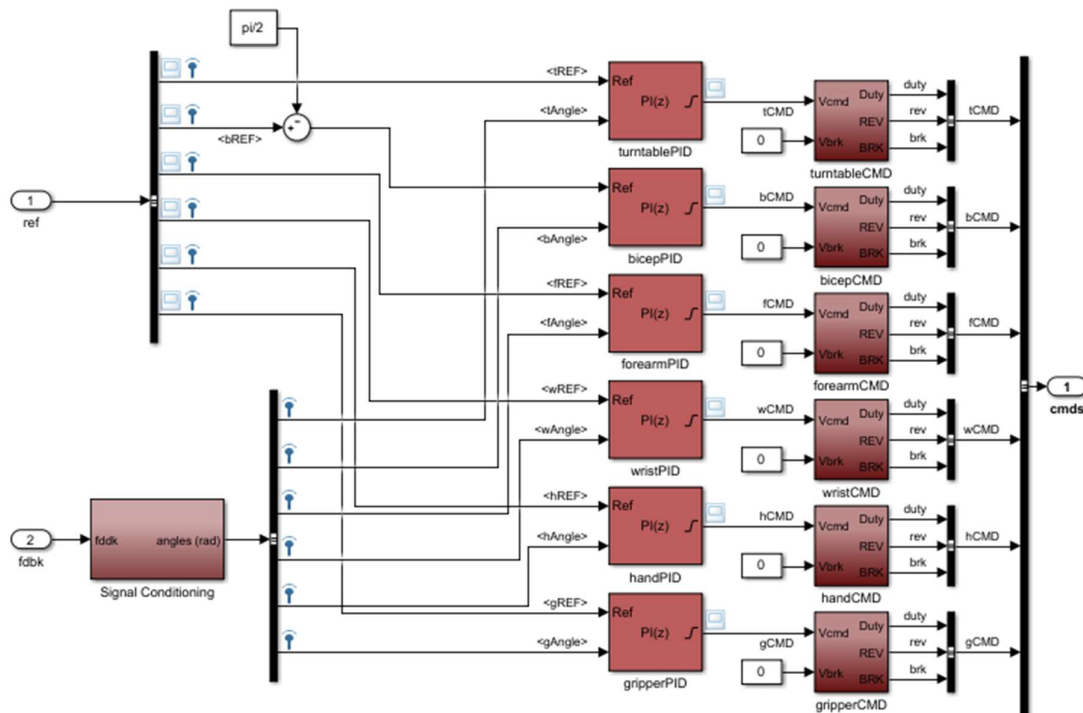


- Robot gồm 6 khớp: Turntable, Bicep, Forearm, Wrist, Hand, Gripper; mỗi khớp dùng động cơ DC (trừ Bicep dùng hai động cơ DC đồng bộ).
- Hệ thống mô phỏng sử dụng Simscape Multibody để mô hình hóa động học — động lực học — trọng lực, quán tính, momen, v.v.
- Bộ điều khiển được cấu trúc theo dạng “multi-loop”: mỗi khớp (joint) có một vòng điều khiển riêng (tách rời), thuận tiện cho việc điều chỉnh tham số PID (hoặc PI) riêng lẻ.



2.3. Phương pháp Tune PID (Multi-Loop)

2.3.1. Tune PID multi-loop



- Do tính phức tạp, coupling động lực học giữa các khớp — một khớp chuyển động có thể ảnh hưởng động học tới các khớp khác (do trọng lượng, quán tính, phản lực, v.v.).
- Bộ PID độc lập từng khớp nếu không cân nhắc coupling có thể dẫn đến dao động, quá vọt (overshoot), không ổn định, hoặc sai số vĩnh viễn lớn.

- Phương pháp multi-loop cho phép phân tách vòng điều khiển khớp, hỗ trợ tune riêng biệt nhưng vẫn xét đến toàn bộ hệ — dễ quản lý hơn khi mở rộng số khớp.

2.3.2. Công cụ và thuật toán sử dụng

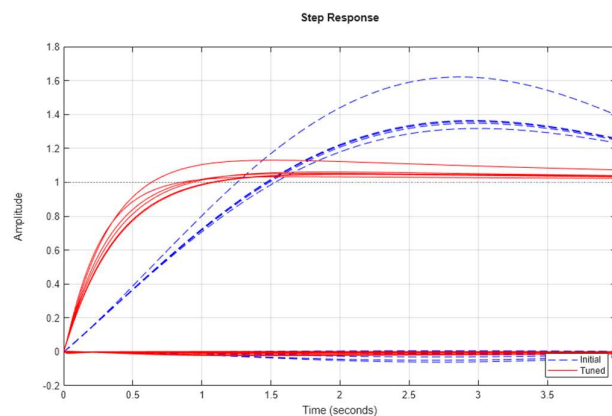
- Trong ví dụ, sử dụng looptune của Simulink Control Design để tự động điều chỉnh các hệ số (gain) cho multi-loop controller.
- Looptune cho phép điều chỉnh đồng thời nhiều vòng, nhằm đảm bảo cả ổn định (robust stability) và đáp ứng (performance) phù hợp. Đây là cách tiếp cận hiện đại, hơn so với tuning thủ công từng khớp truyền thống.

2.3.3. Các bước thực hiện

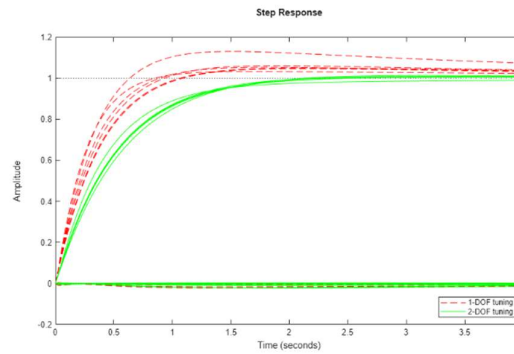
1. Xây dựng mô hình robot đầy đủ trong Simscape Multibody, bao gồm động lực học, quán tính, khớp, khớp servo/DC motor.
2. Thiết lập controller: mỗi khớp một PID/PI loop trong Simulink.
3. Sử dụng looptune — nhập yêu cầu (performance objectives, ổn định, bandwidth, độ lệch cho phép, v.v.) rồi để thuật toán tìm gain thích hợp.
4. Sau khi tuning xong, mô phỏng lại toàn hệ với tín hiệu tham chiếu (reference trajectory) — ví dụ tín hiệu step, hoặc quỹ đạo góc khớp theo thời gian.

Đánh giá đáp ứng: overshoot, thời gian tới trạng thái ổn định (settling time), steady-state error, dao động, coupling ảnh hưởng,...

2.3.4. Kết quả



- PID tuning đã cải thiện rõ rệt chất lượng điều khiển ở tất cả các khớp. Robot sẽ vào vị trí nhanh hơn, ít rung, và an toàn hơn khi thực thi các tác vụ pick-place.

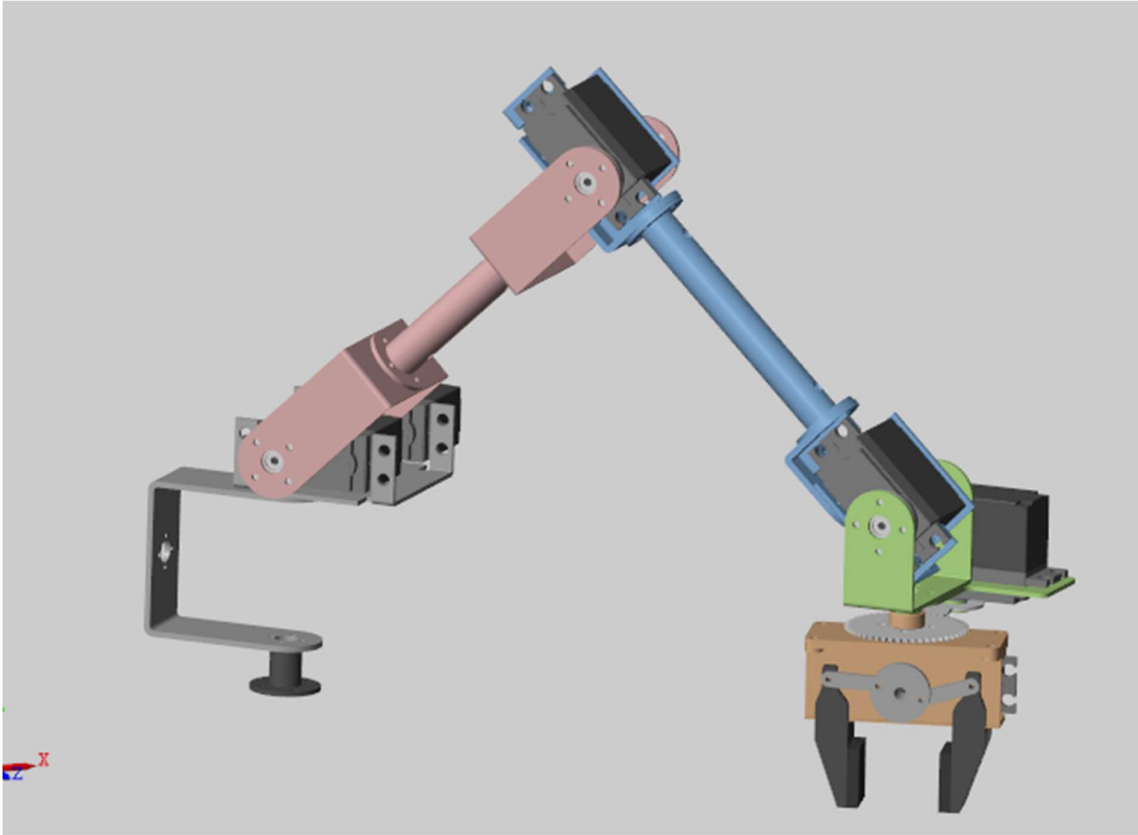


- Trong cấu trúc 2-DOF, bộ điều khiển được chia thành nhánh feedforward + feedback, giúp giảm overshoot và tăng tốc độ đáp ứng.

2.4. Ưu điểm & Hạn chế

- Ưu điểm
 - Tự động & nhanh: looptune giúp giảm đáng kể thời gian thử-sai so với tuning thủ công từng khớp.
 - Robust & đồng nhất: Khi được thiết lập đúng objectives, controller sau tune đảm bảo ổn định cho toàn hệ, không chỉ riêng từng khớp.
 - Dễ mở rộng: Với nhiều khớp hơn, vẫn dùng cấu trúc multi-loop, thuận lợi cho việc quản lý controller.
- Hạn chế & thách thức
 - Mô hình phụ thuộc chính xác: Việc tune hiệu quả phụ thuộc vào mô hình động lực học — quán tính, ma sát, trọng lượng, phản lực — nếu mô hình sai lệch so với thực tế, hiệu năng thực sẽ giảm.
 - Không xử lý phi tuyến nặng / coupling phi tuyến mạnh: PID linear + multi-loop có thể kém hiệu quả nếu robot hoạt động với vận tốc cao, gia tốc lớn, tải thay đổi, hoặc coupling quá mạnh. Trong các hệ phi tuyến phức tạp, có thể cần controller nâng cao hơn.
 - Giới hạn ở cấu hình “decentralized loops”: Trong các hệ có coupling mạnh, việc tách riêng các loop sẽ mất phần hiệu quả phối hợp — đôi khi cần controller đa biến (MIMO) hoặc adaptive / học máy.

III. Mô phỏng thực hiện thao tác xếp hộp vào pallet theo ma trận 3×3



3.1. Cấu hình bài toán

- Vị trí pick cố định.
- Vị trí place là một lưới 3×3.
- Có 3 layer → tổng 27 vật.
- Mỗi vật cần một chu kỳ: PICK → PLACE → trở về cao.

3.2. Quy tắc di chuyển

- Mỗi vật tuân theo chu kỳ:
 - PICK
 - Đi tới điểm trên cao (avoid collision).
 - Hạ xuống điểm pick.
 - Đóng kẹp.
 - Nâng lên lại.
 - PLACE
 - Đi đến điểm trên cao của ô place.
 - Hạ xuống đúng vị trí.
 - Mở kẹp.

- Trở về điểm cao.
- Ý nghĩa:
 - Mọi chuyển động đều tuân thủ quy trình tránh va chạm.
 - Đảm bảo an toàn cho mô phỏng và robot thật.

3.3. Sinh vị trí cho 27 ô trong lưới

- Hệ toạ độ lưới gồm:
 - origin
 - bước theo cột (dx)
 - bước theo hàng (dy)
 - offset theo layer (dz)
- Robot sẽ scan theo thứ tự:
 - layer $0 \rightarrow 1 \rightarrow 2$
 - trong mỗi layer: row $0 \rightarrow 2$
 - mỗi row: col $0 \rightarrow 2$
- Ý nghĩa:
 - Logic đơn giản, có thể thay đổi cách duyệt (zigzag, snake pattern).
 - Dễ mở rộng lên $4 \times 4 \times N$ hoặc pallet thực tế.

3.4. Tích lũy vào quỹ đạo tổng

- Mỗi lệnh Move tạo:
 - một đoạn thời gian
 - một đoạn giá trị joint
- Các đoạn này được nối lại tạo thành refAngles.
- Ý nghĩa:
 - Đây là tín hiệu đầu vào cho controller PID/Computed-Torque/MPC trong Simulink.
 - Đảm bảo quỹ đạo mượt theo thời gian.

3.5. Kết quả đầu ra

- Quỹ đạo cuối cùng gồm:
 - Dãy thời gian t_all (liên tục).
 - Dãy góc khớp q_all tương ứng.
 - Dim = 6 (5 joint + gripper).

- Được lưu vào workspace để mô phỏng.

3.6. Video mô phỏng hoạt động



3.7. Đánh giá thuật toán

- Ưu điểm
 - IK giải nhanh và chính xác theo hình học.
 - Không dùng phương pháp số → không bị phân kỳ.
 - Cấu trúc Move rất đơn giản nhưng hiệu quả cao.
 - Quỹ đạo pick-place rõ ràng, đảm bảo tránh va chạm.
 - Dễ mở rộng thành palletizer thực tế.
- Nhược điểm
 - Không tính đến giới hạn joint → cần thêm bộ kiểm tra.
 - Không tính động lực học → controller phải tự đảm bảo ổn định.

3.8. Kết luận

- Thuật toán giải động học nghịch và sinh quỹ đạo pick-place $3 \times 3 \times 3$ đã thể hiện:
 - Khả năng định vị TCP chính xác nhờ giải hình học.
 - Chuyển động linh hoạt và an toàn nhờ cấu trúc Move theo thời gian.
 - Hoàn thiện chu trình pick-place cho 27 vật theo 3 tầng.
 - Dễ dàng tích hợp vào Simulink hoặc điều khiển robot thật.

PHẦN II: URSim: lập trình UR3 gấp vật và xếp thành hàng – cột.

I. Giới thiệu robot UR3 và bài toán



- Robot sử dụng trong bài là UR3 – robot 6 bậc tự do của Universal Robots, tải trọng nhỏ, phù hợp cho các bài thực hành gấp – đặt và xếp pallet.
- Trong đề tài này, UR3 được dùng để:
 - Nhận gói từ vị trí gấp cố định.
 - Xếp các gói vào vùng pallet theo mẫu hình hộp (Pattern: Box).
 - Chu trình được lặp lại 27 lần, tương ứng với số lượng sản phẩm cần xếp.
- Chương trình được xây dựng và mô phỏng trên URSim/PolyScope, sử dụng các lệnh chuyển động cơ bản MoveJ, MoveL cùng với các lệnh điều khiển I/O và cấu trúc lặp.

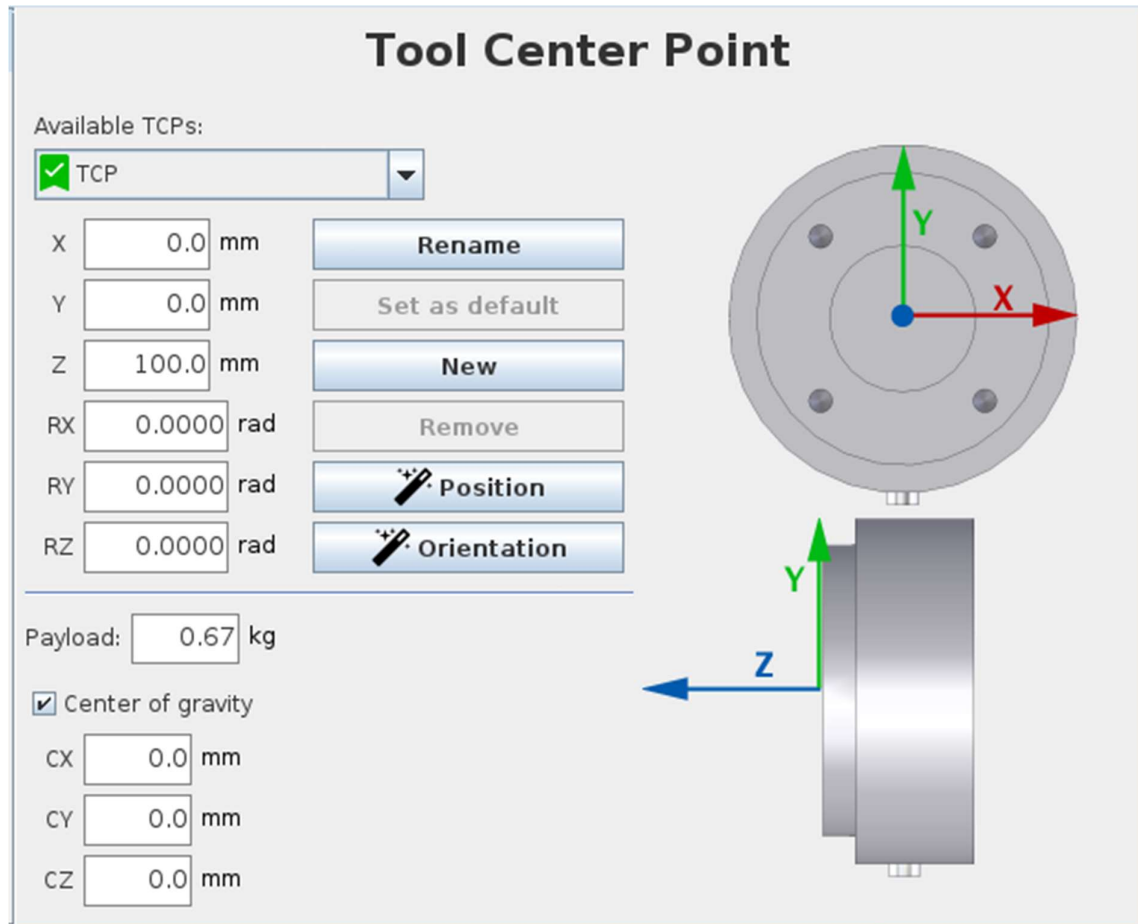
II. Cấu hình chuyển động robot

2.1. Tốc độ và gia tốc

- Để đảm bảo robot chuyển động êm, an toàn nhưng vẫn đủ nhanh cho thực hành, tốc độ và gia tốc được đặt như sau:
 - Tốc độ MoveJ: 0.25–0.30 m/s

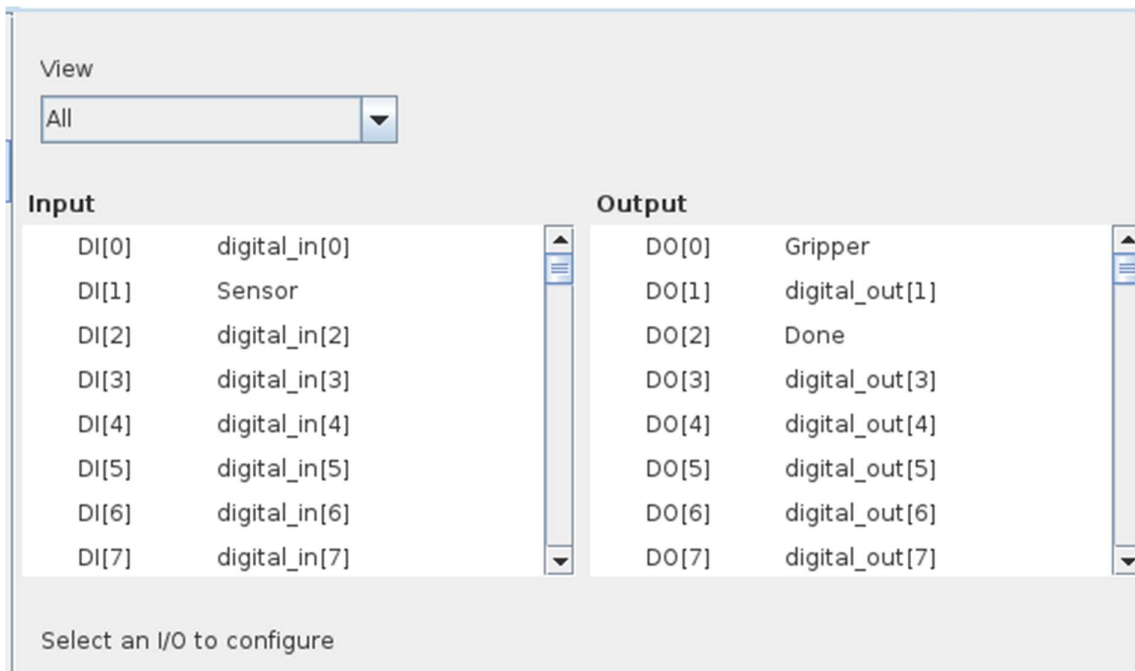
- Tốc độ MoveL: 0.25–0.30 m/s
- Gia tốc (chung): 0.5 m/s²
- Các giá trị này được cấu hình trực tiếp trong các lệnh MoveJ và MoveL trong PolyScope. Tốc độ không quá cao nên robot không bị rung mạnh và tránh va chạm khi làm việc gần người.

2.2. Tool Center Point (TCP)



- Tool của robot là kẹp song song, được gắn trực tiếp vào mặt bích (flange).
- TCP được định nghĩa tại tâm đầu kẹp, dịch dọc trục Z của flange:
 - $TCP = (0, 0, 0.1 \text{ m})$ tính từ flange
- Việc chuẩn hóa cùng một hướng TCP cho toàn bộ waypoint giúp đơn giản hóa việc dạy điểm và đảm bảo sản phẩm được gấp – đặt theo phương thẳng đứng.

2.3. Cấu hình tín hiệu vào/ra (I/O)

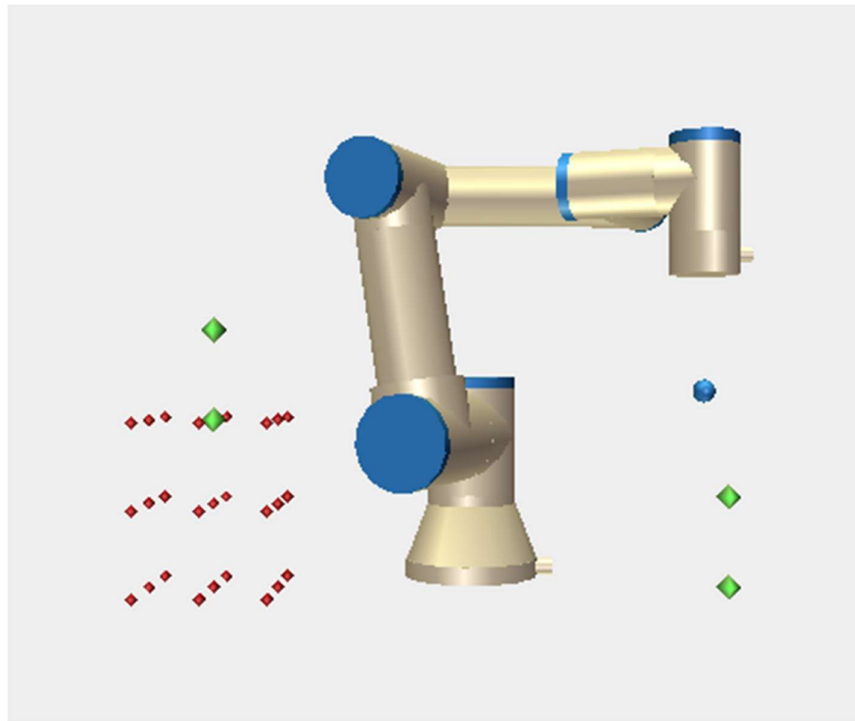


Hệ thống điều khiển sử dụng các I/O số (digital) như sau:

- DO[0] – Gripper
 - DO[0] = ON → Kẹp đóng, giữ phôi.
 - DO[0] = OFF → Kẹp mở, nhả phôi.
- DI[1] – Cảm biến phôi
 - DI[1] = HI → Có phôi tại vị trí gắp, cho phép robot bắt đầu chu trình.
 - DI[1] = LO → Không có phôi, robot chờ.
- DO[2] – Báo hoàn thành
 - DO[2] = ON khi toàn bộ quá trình xếp pallet kết thúc.
 - DO[2] = OFF trong lúc chương trình đang chạy.

Việc tách I/O rõ ràng giúp dễ dàng kết nối robot với hệ thống cấp phôi và đèn/báo còi ở ngoài.

2.4. Thiết lập các điểm làm việc (Waypoint)



2.4.1. Hệ trục và đơn vị

- Tất cả các tọa độ được dạy trong Base frame của robot.
- Hướng TCP cho tất cả điểm: $(R_x, R_y, R_z) = (\pi, 0, 0)$.

2.4.2. Các điểm khu vực gấp

Tên điểm	X (mm)	Y (mm)	Z (mm)	Ghi chú
HomePOS	200	200	200	Vị trí “home”, tư thế an toàn trước và sau chương trình
Waypoint_1	300	200	120	Điểm tiếp cận vùng gấp (trên phôi)
Pick	300	200	50	Vị trí gấp phôi, gần sát bề mặt

- Chu trình gấp:
 - MoveL HomePOS → Waypoint_1 → Pick → Waypoint_1.

2.4.3. Các điểm khu vực pallet

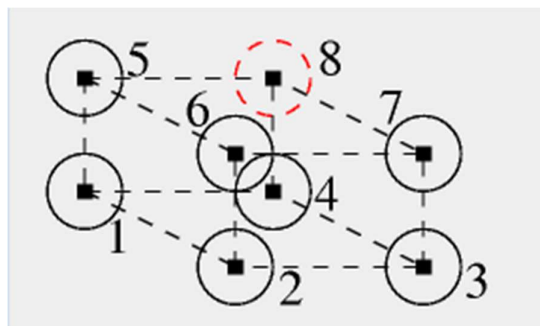
- Điểm tiếp cận và điểm đặt (PalletSequence)

Tên điểm	X (mm)	Y (mm)	Z (mm)	Ghi chú
Approach	300	-200	250	Điểm tiếp cận pallet, cao an toàn

PatternPlace	300	-200	180	Điểm đặt phôi tại ô đang chọn
Exit_	300	-200	250	Rút tay lên lại sau khi đặt

- Các điểm này được dùng trong chương trình con PalletSequence.
- Các điểm Corner – Pattern: Box
- Pattern “Box” được xây dựng từ 8 điểm Corner1_1 ... Corner8_1, tạo thành một hộp bao quanh vùng pallet, dùng cho lệnh Pattern của UR để sinh ra các vị trí đặt theo dạng ma trận.

Tên điểm	X (mm)	Y (mm)	Z (mm)
Corner1_1	300	-200	50
Corner2_1	400	-200	50
Corner3_1	400	-300	50
Corner4_1	300	-300	50
Corner5_1	300	-200	180
Corner6_1	400	-200	180
Corner7_1	400	-300	180
Corner8_1	300	-300	180



- Bảng tọa độ các điểm Corner (mm):
 - Bốn điểm Corner1_1...Corner4_1: mặt dưới của khối box.
 - Bốn điểm Corner5_1...Corner8_1: mặt trên.

- URSim sử dụng 8 điểm này để suy ra các vị trí pallet bên trong (3×3 hoặc 3×n) tùy cấu hình Pattern Box trong phần Pallet.

2.5. Các lệnh chính sử dụng trong chương trình

2.5.1. MoveJ

- Dùng để di chuyển robot giữa các tư thế rời rạc, thường ở vị trí cao, an toàn:
 - Ví dụ: HomePOS, chuyển sang vùng gấp, chuyển sang vùng pallet.
- Đặc điểm:
 - Chuyển động trong không gian khớp (joint space).
 - Đường đi của TCP có thể không thẳng nhưng các khớp đi theo quỹ đạo mượt.
- Trong chương trình: MoveJ được dùng cho các đoạn:
 - Từ HomePOS đến Waypoint_1 (hoặc chiều ngược lại nếu cần).
 - Di chuyển giữa các vùng trước/sau khi xếp pallet.

2.5.2. MoveL

- Dùng ở những đoạn cần đi thẳng:
 - Hạ xuống vị trí gấp (Waypoint_1 → Pick).
 - Hạ xuống/nhấc lên tại vùng pallet (Approach → PatternPlace → Exit_).
- Ưu điểm:
 - Đảm bảo TCP đi theo đường thẳng, tránh quệt vào phôi hoặc thành pallet.

2.5.3. SetDO

- Set DO[0] = On/Off:
 - ON: kẹp đóng, gấp phôi.
 - OFF: kẹp mở, thả phôi.
- Set DO[2] = On: bật tín hiệu báo hoàn thành sau khi Loop kết thúc.

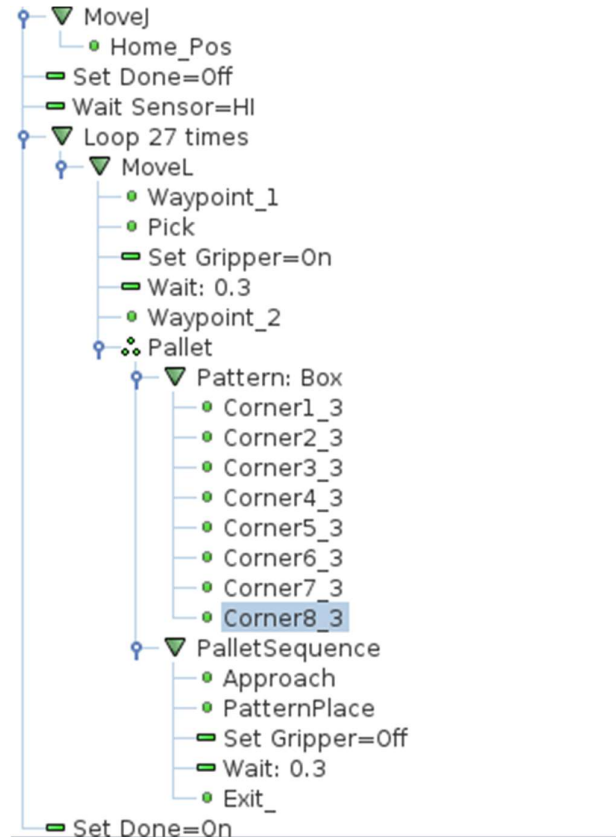
2.5.4. Wait

- Wait Sensor = HI: chương trình chờ cho đến khi DI[1] = HI, tức là cảm biến báo có phôi.
- Wait: 0.3 s sau khi kẹp/nhả:
 - Cho thời gian cơ cấu kẹp đóng/mở hoàn toàn.
 - Tránh robot di chuyển quá nhanh khi phôi chưa kịp được giữ chắc.

2.5.5. Loop

- Lệnh Loop 27 times:
 1. Chu trình gấp – xếp được lặp lại 27 lần, tương ứng với số lượng sản phẩm cần xếp (ví dụ 3 hàng $\times$ 3 cột $\times$ 3 lớp).
- Trong mỗi vòng lặp:
 1. Robot gấp phôi tại vùng gấp.
 2. Gọi chương trình con Pallet để đặt phôi vào vị trí tiếp theo trên pallet.

2.6. Cấu trúc chương trình (cây chương trình)



- Cây chương trình trên PolyScope thể hiện như hình, gồm các khối chính:
 - Home_Pos
 - Robot về vị trí home.
 - Set Done = Off.
 - Wait Sensor = HI
 - Chờ cảm biến báo có phôi (DI[1]).
 - Loop 27 times
 - Mỗi vòng lặp gồm:
 - MoveL
 - Waypoint_1
 - Pick

- Set Gripper = On (DO[0] = On)
- Wait 0.3 s
- Waypoint_1 (rút tay)
- Pallet
 - Pattern: Box
 - Corner1_3 ... Corner8_3
 - PalletSequence
 - Approach
 - PatternPlace
 - Set Gripper = Off (nhả phôi)
 - Wait 0.3 s
 - Exit_
- Set Done = On
 - Sau khi Loop hoàn tất, bật DO[2] để báo hoàn thành.

2.7. Video mô phỏng hoạt động



2.8. Nhận xét

- Việc lựa chọn tốc độ 0.25–0.3 m/s và gia tốc 0.5 m/s² giúp robot UR3 hoạt động êm, ổn định, phù hợp cho môi trường phòng thí nghiệm.
- TCP được định nghĩa đơn giản (0,0,0.1 m) theo trục Z của flange nên việc dạy điểm và mô phỏng dễ dàng, tránh nhầm lẫn.
- Hệ thống I/O rõ ràng (DO[0] – kẹp, DI[1] – cảm biến, DO[2] – báo hoàn thành) giúp chương trình dễ tích hợp với hệ thống cấp phôi và hệ thống giám sát bên ngoài.
- Các waypoint được bố trí trong vùng làm việc an toàn của UR3 (xấp xỉ bán kính 300–400 mm), không gây lỗi *unreachable* và đảm bảo robot không va chạm với bàn/pallet.

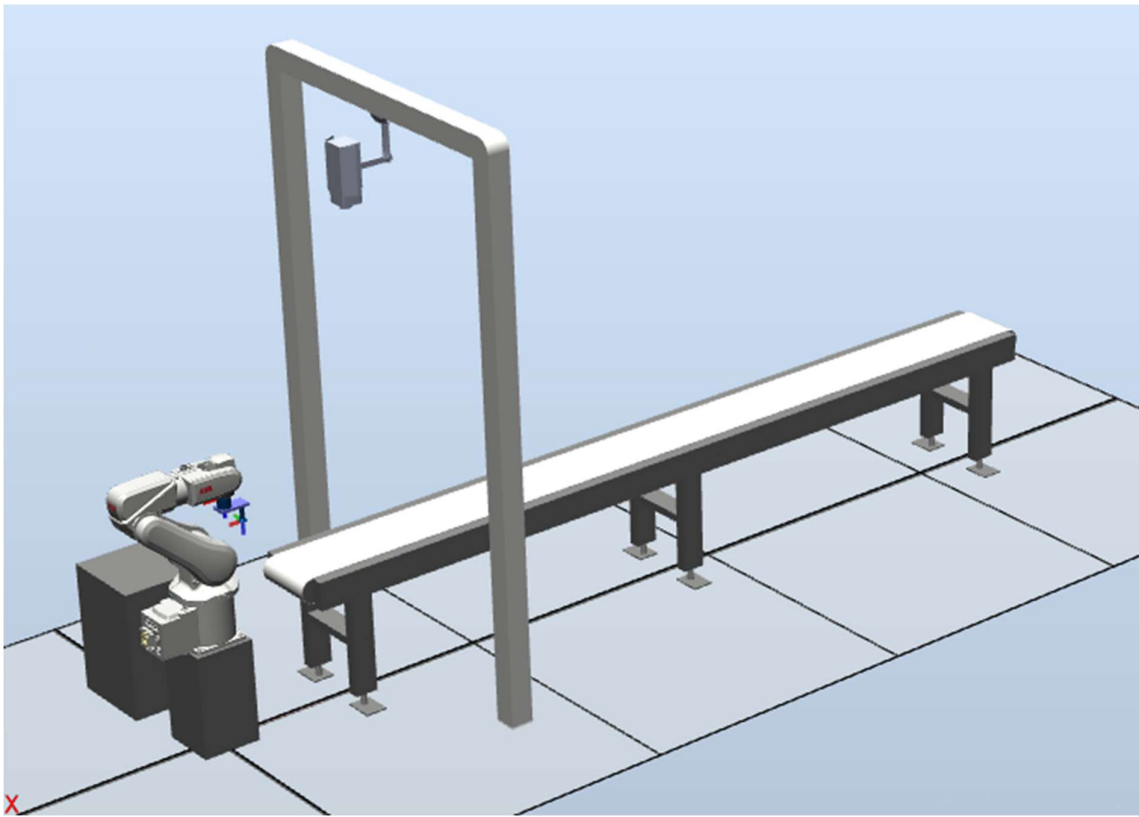
PHẦN III: RobotStudio: mô phỏng thực hiện nhiệm vụ xếp pallet.

I. Mục tiêu hệ thống

- Xây dựng mô hình mô phỏng 3D trong RobotStudio gồm:
 - Robot ABB IRB120 (tool hút chân không).
 - Băng tải tạo phôi tự động.
 - Cổng camera (giá treo + camera) nhìn từ trên xuống băng tải.
- Thiết kế chuỗi SmartComponent:
 - *Conveyor Belt* sinh hộp di chuyển trên băng.
 - *RB_SUCK* mô phỏng hút/thả phôi của robot.
 - *CAMERA* chụp ảnh vùng băng tải và lưu ra file PNG.
- Lập trình:
 - RAPID module VisionPick trên controller ABB:
 - Giao tiếp socket TCP/IP với máy tính.
 - Gửi lệnh “CAPTURE” cho máy tính xử lý ảnh.
 - Nhận DX, DY, ANG (mm, deg) của hộp so với mép trái băng tải.
 - Tính toán offset rotarget, gấp hộp và xếp vào pallet $3 \times 3 \times N$.
 - Chương trình Python:
 - Nhận lệnh từ ABB qua socket.
 - Đọc ảnh pic.png (do SmartComponent CAMERA chụp).
 - Xử lý ảnh tìm băng tải, mép trái và hộp.
 - Tính DX, DY, ANG và gửi lại cho ABB.
 - Có giao diện Tkinter hiển thị kết quả & log.

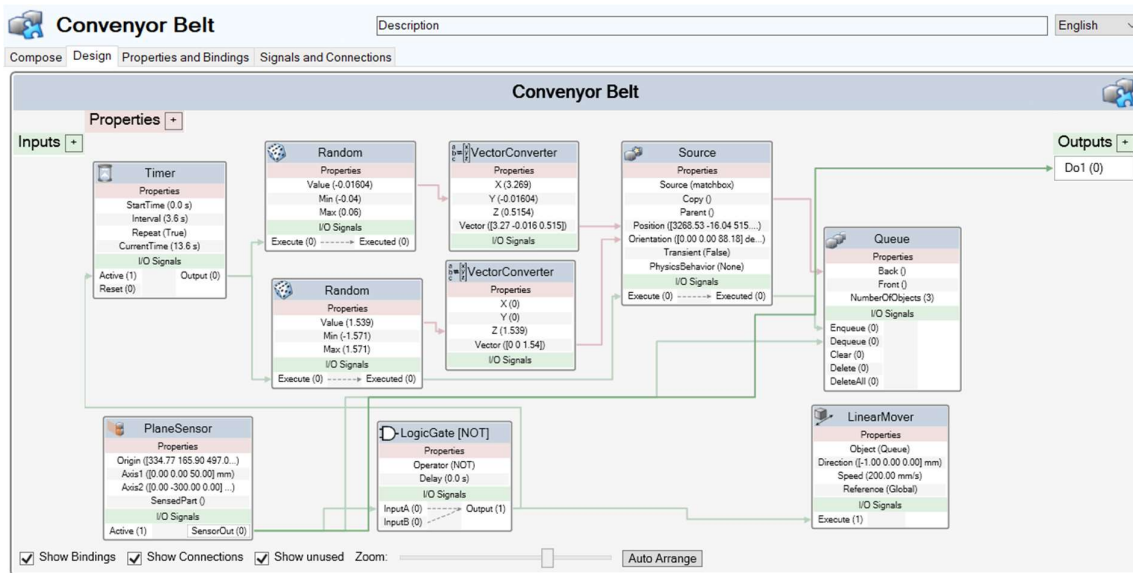
II. Mô hình mô phỏng RobotStudio

2.1. Layout cơ khí



- IRB120 đặt cạnh đầu băng tải.
- Băng tải dài, mặt băng màu trắng, khung biên màu xám.
- Khung cổng chữ U đặt phía trên băng, giữa cổng gắn camera ảo (SmartComponent).
- Vùng làm việc:
 - Vùng pick: trên băng, ngay dưới camera.
 - Vùng place: pallet/khối đặt hộp ở phía cạnh bên robot.

2.2. SmartComponent “Conveyor Belt”



Các block chính:

1. Timer

- StartTime = 0 s.
- Interval ~ 3.6 s (chu kỳ sinh phôi mới).
- Repeat = True.
- Output → kích hoạt khối *Random* và *Source*.

2. Random (vị trí dọc băng – X) + VectorConverter

- Random1: tạo giá trị ngẫu nhiên nhỏ cho tọa độ X, Z của phôi khi sinh.
- VectorConverter1: ghép 3 thành phần X, Y, Z thành vector tọa độ.
- Đầu ra → Source (vị trí đặt hộp).

3. Random (vị trí ngang – Y) + VectorConverter

- Random2: tạo độ lệch nhỏ theo phương Y.
- VectorConverter2: ghép vector offset.
- Đầu ra → Source (offset cuối cùng).

4. Source (matchbox)

- Source object: model “matchbox”.
- Mỗi lần Timer kích → tạo mới 1 hộp trên băng, vị trí & orientation có nhiều ngẫu nhiên.
- Đầu ra nối vào Queue.

5. Queue

- Giữ danh sách tất cả hộp trên băng.
- Thực hiện enqueue/dequeue cho LinearMover.

6. LinearMover

- Object: Queue.
- Direction: (1,0,0) – dọc theo băng.
- Speed: ~200 mm/s.
- Điều khiển chuyển động của các hộp theo hàng trên băng tải.

7. PlaneSensor

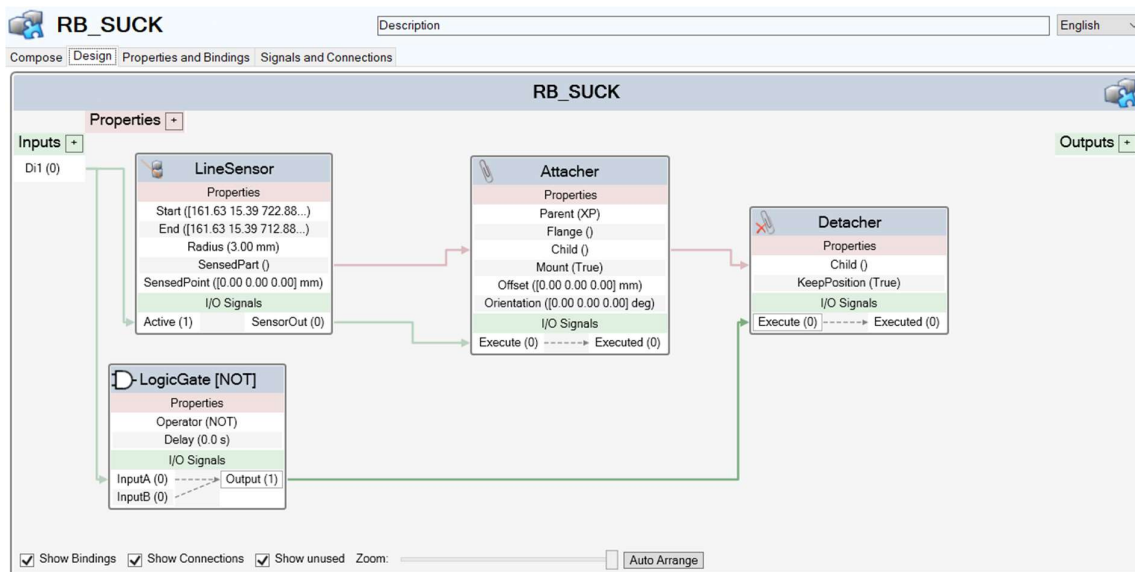
- Đặt ở đầu băng, vị trí ngay trước vùng robot pick.
- Khi hộp đi vào vùng cảm biến → SensorOut = 1.
- Output này được xuất ra DO1 của SmartComponent, rồi nối sang DI “Sensor” của robot.

8. LogicGate [NOT]

- Có thể dùng để đảo hoặc xử lý logic thêm nếu cần.

Kết quả: SmartComponent Conveyor Belt tự động sinh hộp, di chuyển trên băng và xuất tín hiệu “có hộp” sang robot.

2.3. SmartComponent “RB_SUCK”



1. LineSensor

- Dải cảm biến hình trụ đi qua mỏ hút (tool XP).
- Khi phôi nằm trong vùng → SensorOut = 1.

2. LogicGate [NOT]

- Nhận DI “suck” từ robot, xử lý logic (gần như: khi DO robot bật hút → kích Attacher).

3. Attacher

- Parent: XP (tool robot).

- Child: phôi được LineSensor “nhìn thấy”.
- Khi Execute = 1 → gắn hộp vào TCP.

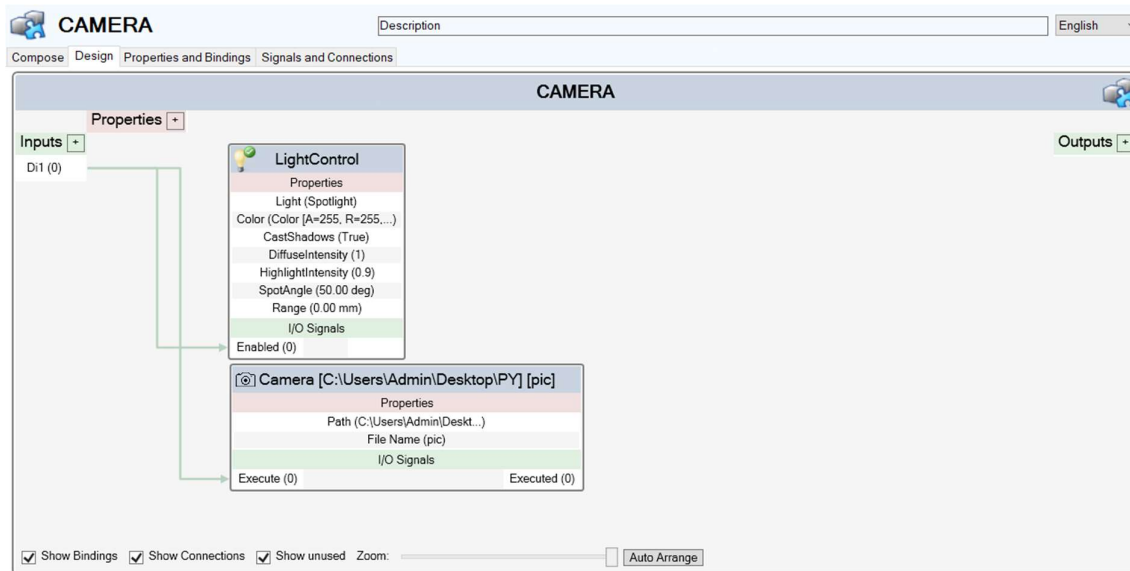
4. Detacher

- Child: đối tượng đang gắn.
- Khi Execute = 1 → thả hộp (không gắn nữa, vẫn giữ vị trí hiện tại).

RB_SUCK nối với DI/DO của robot:

- Robot bật DO suck → Attacher được kích, phôi “đính” vào tool.
- Robot tắt DO suck → Detacher thả phôi xuống pallet.

2.4. SmartComponent “CAMERA”



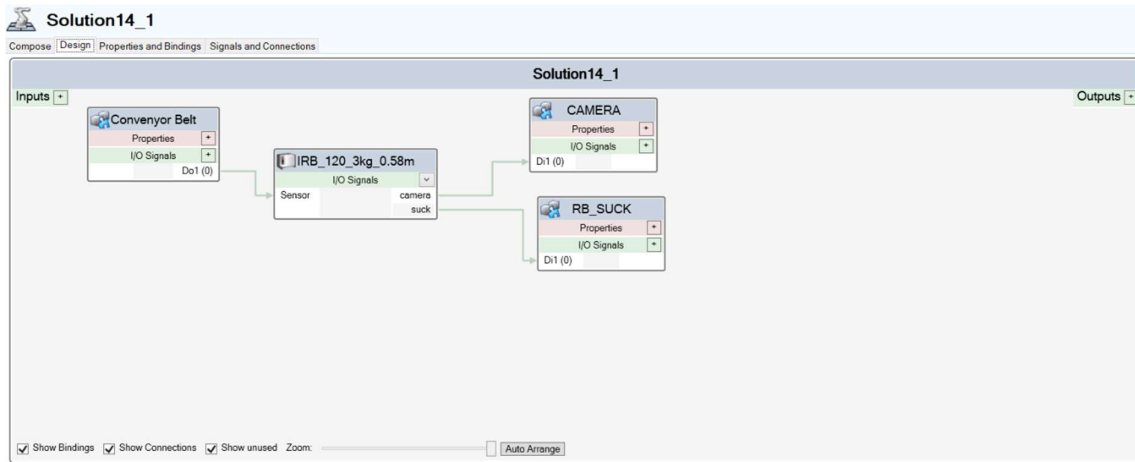
1. LightControl

- Điều khiển spotlight chiếu đúng vùng băng, đảm bảo ảnh rõ ràng.
- Input: DI1 (tín hiệu “camera” từ robot).

2. Camera [path ...\PY][pic]

- Path: thư mục chứa ảnh (C:\Users\Admin\Desktop\PY)
- File Name: pic → file sinh ra là pic.png.
- Khi Execute = 1:
 - Camera SC chụp ảnh viewport đặt trước đó, lưu đúng file PNG này.
- Execute được nối từ DI1 của SmartComponent (lấy từ DO “camera” của robot).

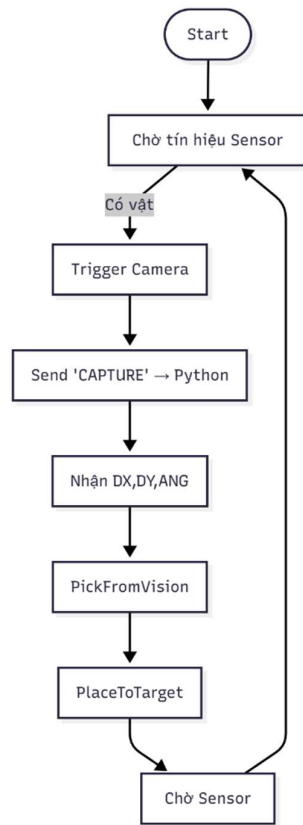
2.5. Kết nối tổng thể Solution



- Conveyor Belt → Output DO1 (0) → nối vào DI “Sensor” của IRB120.
- IRB120:
 - DO “suck” → SmartComponent RB_SUCK (Di1).
 - DO “camera” → SmartComponent CAMERA (Di1).
- Hai SmartComponent CAMERA và RB_SUCK chỉ nhận tín hiệu từ IRB120, không xuất ngược.

III. Thuật toán và chương trình RAPID

Lưu đồ thuật toán



3.1. Mục tiêu của module VisionPick

- Module RAPID VisionPick có nhiệm vụ:
 - Nghe kết nối từ chương trình Python vision client qua socket TCP.
 - Khi cảm biến trên băng tải phát hiện có hộp đi vào vùng camera:
 - Kích camera chụp ảnh và yêu cầu Python xử lý ảnh.
 - Nhận về kết quả dịch chuyển X, Y (mm) và góc quay (deg) của hộp so với hệ tọa độ băng tải.
 - Tính toán quỹ đạo gấp (pick) bám theo kết quả thị giác.
 - Sau đó đưa hộp đến vị trí xếp pallet $3 \times 3 \times N$ theo thứ tự.
 - Toàn bộ chu trình chạy liên tục trong vòng lặp vô hạn cho tới khi dừng chương trình trên robot.

2.4. Chu trình

1. Khởi tạo socket server

- Tạo thiết bị socket server, gán địa chỉ IP + port, đặt ở chế độ lắng nghe.
- Chờ Python kết nối tới, sau khi accept thành công thì gửi log “Python connected...”.

2. Đưa robot về trạng thái an toàn ban đầu

- Đặt tín hiệu hút OFF để chắc chắn không giữ phôi.
- Thực hiện một chuyển động MoveJ về vị trí Home (có wobj và tool phù hợp).

3. Vòng lặp chính

a. Chờ sensor phát hiện hộp

- Dùng lệnh chờ tín hiệu số đầu vào (Digital Input) từ mô-đun băng tải: khi có hộp đi vào vùng cảm biến $\rightarrow$ DI = 1.
- Gửi log “Sensor ON - co vat”.

b. Kích hoạt camera

- Bật tín hiệu điều khiển camera/logic snapshot trong Smart Component (Digital Output).
- Giữ trong một khoảng thời gian ngắn (0.5s) rồi tắt.
- Log “Trigger camera xong”.
- Ở Smart Component CAMERA, tín hiệu này sẽ kích việc lưu ảnh PNG xuống đường dẫn đã cấu hình.

c. Yêu cầu Python xử lý ảnh

- Gửi chuỗi lệnh "CAPTURE" qua socket.
- Python nhận được lệnh, sẽ:
 - Đọc ảnh / frame camera.
 - Tính DX, DY, ANG.
 - Gửi lần lượt ba dòng "DX:...", "DY:...", "ANG:...".

d. Nhận kết quả từ Python

- Gọi thủ tục ReceiveVisionResult, sẽ block cho tới khi nhận đủ ba giá trị số đã chuyển đổi đúng.
- Sau khi có dx, dy, ang, robot log “Đã nhận đầy đủ DX, DY, ANG”.

e. Xác nhận cho Python

- Gửi lại một dòng "OK_VISION" cho Python (chỉ để debug/tracking, không bắt buộc).

f. Thực hiện gấp

- Gọi PickFromVision để tính quỹ đạo pick dựa trên dx, dy, ang và thực hiện MoveJ/MoveL + bật hút.

g. Đặt hộp lên pallet

- Gọi PlaceToTarget:
 - Dựa vào count hiện tại để xác định ô (col,row,layer).

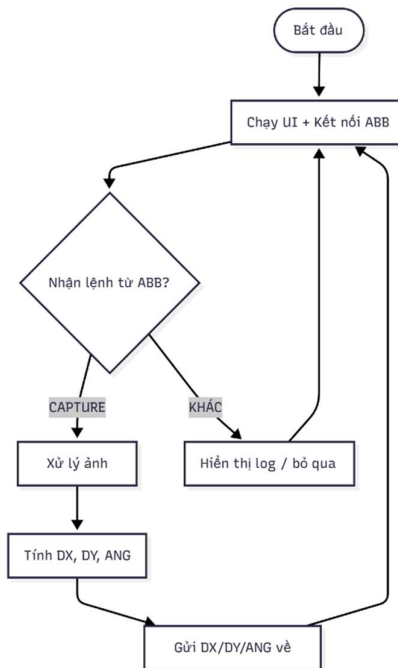
- Tính offset và thực hiện chuyển động đặt phôi.
- Tăng count.

h. Chờ sensor về 0

- Trước khi quay lại đầu vòng, robot chờ sensor = 0 (hộp đã ra khỏi vùng cảm biến).
- Gửi log “Sensor OFF - cho vòng mới”.

IV. Thuật toán xử lý ảnh & chương trình Python

Lưu đồ thuật toán



- Chương trình gồm 3 khối:

1. Phần cấu hình

- Các biến cấu hình nguồn ảnh:
 - Chọn dùng ảnh test (USE_TEST_IMAGE) hay camera (CAM_INDEX).
 - Đường dẫn ảnh (IMG_PATH).
- Các tham số hình học:
 - Bề rộng băng tải thật BELT_WIDTH_MM = 300 mm.

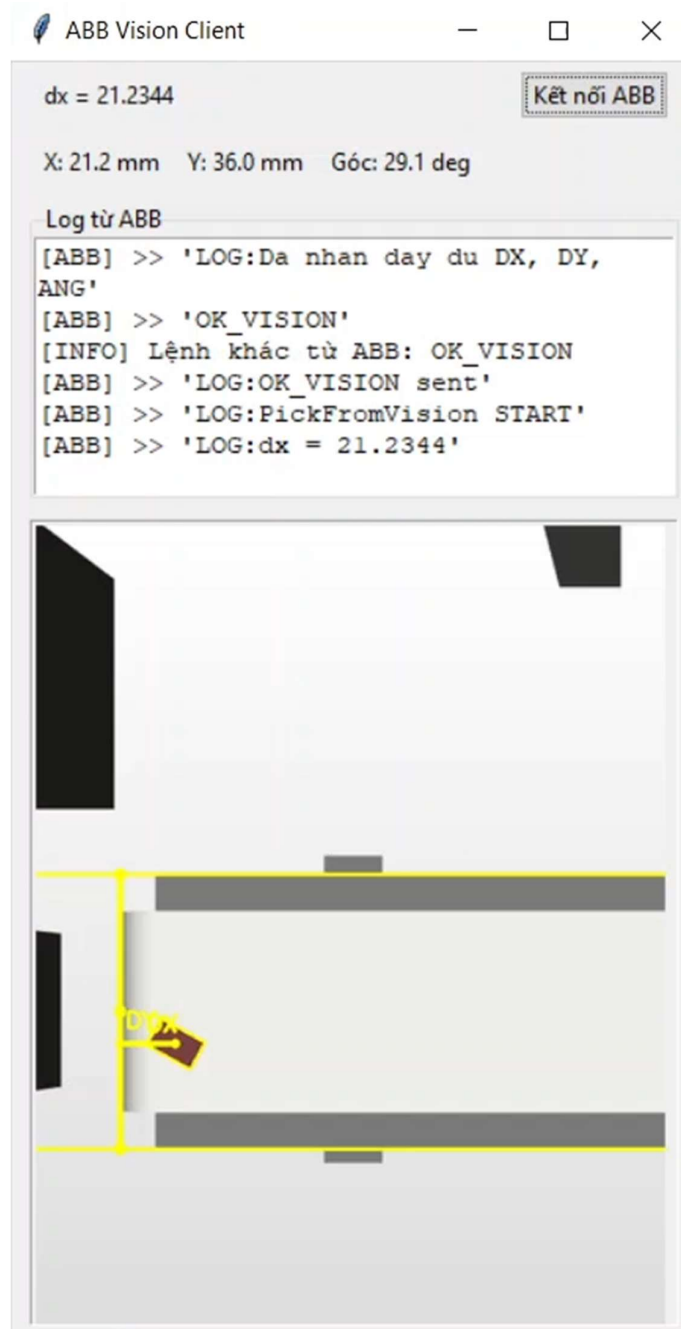
- Tỷ lệ bỏ vùng giữa băng để tránh cạnh box (IGNORE_MIDDLE_MM).
- Tỷ lệ bỏ mép trái (IGNORE_LEFT_RATIO) để loại tường/không gian ngoài.
- Thông số socket:
 - HOST, PORT phải trùng với HOST, PORT bên RAPID.
- Màu vẽ kết quả (DRAW_COLOR) để overlay lên ảnh.

2. Nhóm hàm xử lý ảnh (vision)

- Các hàm:
 - Tìm mép trên/dưới băng tải.
 - Tìm mép trái băng tải.
 - Tìm contour hộp trên băng.
 - Gộp pipeline thành hàm xử lý ảnh tổng thể, trả về X_mm, Y_mm, angle.

3. Lớp giao diện & socket (VisionClientUI)

- Tạo UI bằng Tkinter.
- Quản lý kết nối socket đến ABB.
- Nhận lệnh từ ABB (LOG, CAPTURE, QUIT...).
- Khi nhận "CAPTURE" thì:
 - Gọi khối xử lý ảnh.
 - Cập nhật GUI.
 - Gửi DX, DY, ANG lại cho ABB.



V. Quy trình hoạt động tổng thể

5.1. Quy trình hoạt động

1. Khởi tạo mô phỏng:

- Chạy Solution trong RobotStudio:
 - Conveyor Belt sinh hộp.

- Camera & RB_SUCK SC đã kết nối I/O.
- Chạy RAPID module VisionPick.main.
- Chạy Python VisionClientUI và nhấn nút “Kết nối ABB”.

2. Khi hộp đi vào vùng cảm biến:

- PlaneSensor trên băng → DO1 = 1 → robot DI sensor = 1
- RAPID:
 - Chờ WaitDI sensor,1 xong.
 - Bật DO camera → CAMERA chụp pic.png.

3. ABB yêu cầu Python xử lý ảnh:

- Gửi log: LOG:Trigger camera xong.
- Gửi lệnh: CAPTURE\n.

4. Python xử lý ảnh:

- Nhận CAPTURE, đọc ảnh pic.png.
- Chạy pipeline: Canny → find belt → find left edge → find box → tính X_mm, Y_mm, ang.
- Gửi lại:
 - DX:<X_mm>\n
 - DY:<Y_mm>\n
 - ANG:<ang>\n

5. ABB nhận kết quả và pick:

- ReceiveVisionResult parse chuỗi, lưu vào dx,dy,ang.
- Gửi lại OK_VISION.
- Tính robtarget pick:
 - tBase = Offs(Target_30, dx, -dy, 10.89).
 - tPick = RelTool(tBase, ..., \Rz:=ang).
- MoveJ đến trên cao, MoveL xuống pick, bật hút.

6. ABB xếp hộp lên pallet 3×3×N:

- Từ biến count xác định hàng/cột/layer.
- Offset từ origin → tPlace.

- Đặt hộp, tắt hút, tăng count.

7. Chu kỳ lặp lại cho những hộp tiếp theo.

5.2. Video mô phỏng hoạt động



5.3. Đánh giá & hướng phát triển

- **Ưu điểm**
 - Tự động hóa hoàn toàn chuỗi mô phỏng:
 - Hộp sinh ngẫu nhiên vị trí và góc.
 - Vision tự tính toán offset, robot gấp chính xác theo tọa độ ảnh.
 - Kiến trúc rõ ràng:
 - Tách biệt giữa:
 - Mô phỏng cơ khí (RobotStudio + SmartComponent).
 - Điều khiển robot (RAPID).
 - Vision & UI (Python).
 - Dễ mở rộng:
 - Chỉ cần thay đổi hàm analyze_image để dùng YOLO hoặc thuật toán khác.
 - Dễ chuyển từ ảnh giả lập sang camera thật bằng cách đổi USE_TEST_IMAGE.
- **Hạn chế**
 - Các tham số như:
 - BELT_WIDTH_MM, IGNORE_MIDDLE_MM, XLEFT_OFFSET_PX, hệ số lọc contour...
 - Được chọn thực nghiệm, cần tinh chỉnh thêm nếu thay đổi layout.
 - Vision hiện tại chỉ xử lý 1 loại hộp với kích thước tương đối cố định.

- Hệ thống chưa xử lý trường hợp có nhiều hộp chồng nhau trong vùng camera.
- **Hướng phát triển**
 - Thay thuật toán tìm contour bằng detection mạng học sâu (YOLOv8, YOLOv11...).
 - Hiệu chỉnh lại transform pixel $\rightarrow$ mm bằng ma trận homography thay vì giả định tỷ lệ tuyến tính.
 - Bổ sung:
 - Chức năng dừng khẩn theo lỗi vision.
 - Giao diện Python hiển thị history nhiều lần chụp + thống kê sai số.